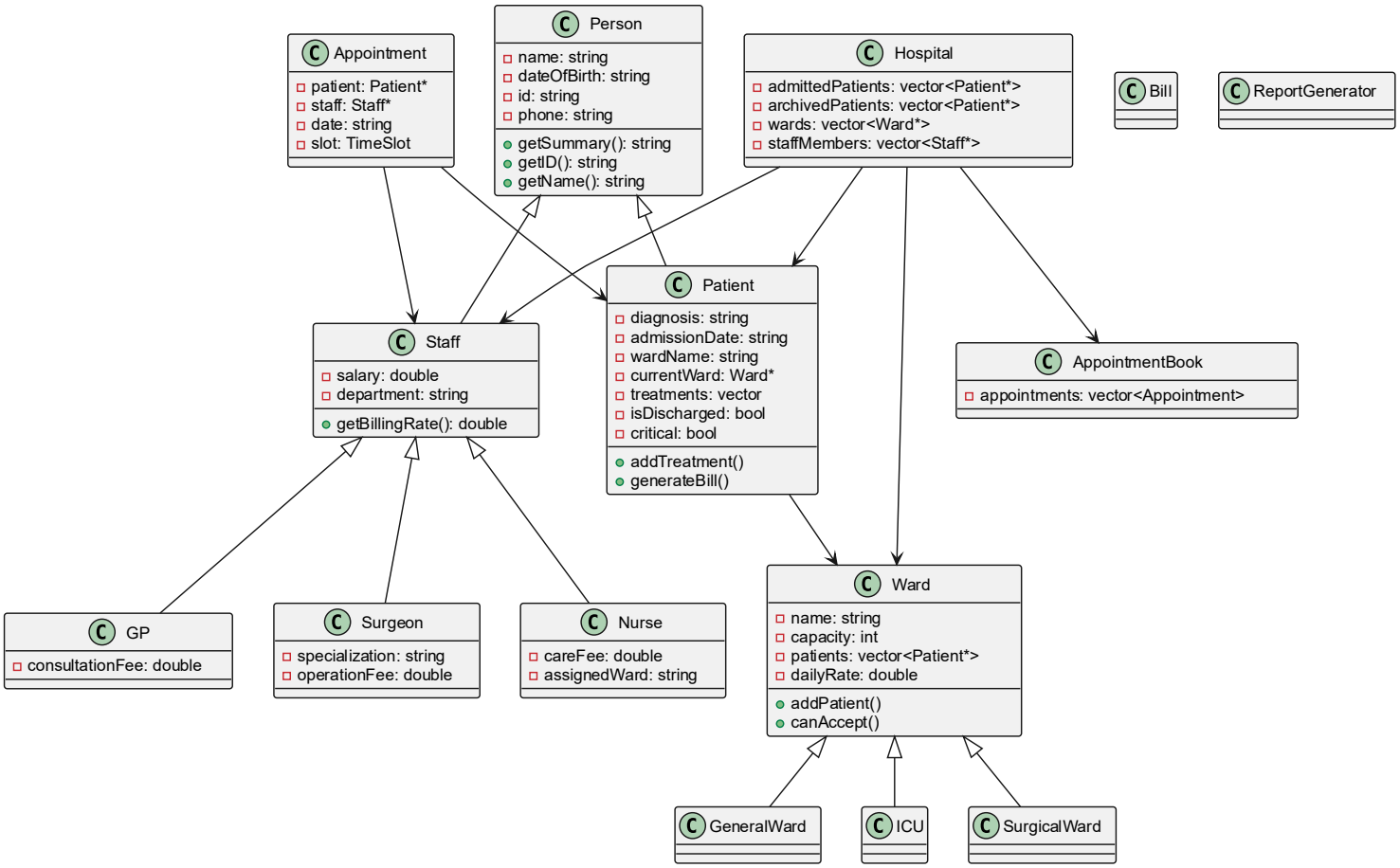
Medicore

Assignment 2
Ali Awj – 32455

1. UML Diagram



2. Class Diagram (Write-up)

Class Diagram Explanation

The system is designed using an object-oriented approach with a strong emphasis on inheritance, polymorphism, and composition.

At the top of the hierarchy is the Person class, which serves as a base class for both Staff and Patient. The Staff class is further specialized into GP, Surgeon, and Nurse, each implementing their own version of getBillingRate().

The Patient class contains treatment records and maintains a relationship with a Ward. The Ward class acts as an abstract base class, with specialized implementations such as GeneralWard, ICU, and SurgicalWard, each enforcing its own admission rules.

The Hospital class serves as the central controller, managing:

- Patients (admitted and archived)
- Staff members
- Wards
- Appointment scheduling via AppointmentBook

The Appointment class represents the association between a Patient and a Staff member, while the AppointmentBook ensures scheduling constraints.

The ReportGenerator class provides utility functions for filtering, sorting, and revenue calculations.

3. Design Decisions

1. Template Method Pattern for Ward Admission

The Ward::addPatient() function defines the general admission workflow, while delegating specific eligibility checks to the virtual canAccept() function. This ensures consistent enforcement of capacity constraints while allowing specialized wards like ICU and SurgicalWard to define their own admission criteria.

2. Use of Polymorphism Instead of dynamic_cast

The system uses pure virtual functions such as getBillingRate() to achieve runtime polymorphism. This eliminates the need for dynamic_cast, ensuring cleaner and more maintainable code while leveraging the virtual table mechanism.

3. Value Semantics for Patient Data

The Patient class uses std::vector to store treatments. This ensures automatic memory management and avoids manual deletion, reducing the risk of memory leaks.

4. Lambda-Based Filtering and Sorting

The use of std::function and lambda expressions enables flexible querying and sorting of patients and staff. This design decouples the filtering logic from the Hospital class, allowing new behaviors to be introduced without modifying existing code.

5. Operator Overloading for Billing

The Bill class overloads operators such as + and == to simplify financial computations. This improves readability and allows intuitive combination and comparison of billing objects.

6. Separation of Concerns

Each class has a clearly defined responsibility:

- Hospital → Management
- Patient → Medical data
- Ward → Admission logic
- ReportGenerator → Analytics

This improves modularity and maintainability.

4. Memory Ownership Diagram (Final)

Memory Ownership Table

Resource	Owner	Allocation	Deletion
Ward Objects	Hospital	main (new)	Hospital destructor
Staff Objects	Hospital	main (new)	Hospital destructor
Patient Objects	Hospital	admit() (new)	Hospital destructor
Patient (main test cases)	main()	new	In main
Treatment Data	Patient	vector	automatic
Appointment Objects	AppointmentBook	vector	automatic

Ownership Rules

Hospital OWNS:

- Staff
- Wards
- Admitted & Archived Patients

Patient OWNS:

- Treatments (via vector)

AppointmentBook OWNS:

- Appointments (by value)